00 - Data Preparation

Purpose and Scope

This notebook constitutes the absolute ground-zero fundamentally driving the **data calibration and hardware load profiling** phase of the entire thesis architecture. Manipulating the raw dispatch logs of the Alibaba PAI 100K GPU cluster, it acts as the primary engineering backbone where dynamic utilization features—mapping instantaneous cluster stress levels across time—are mathematically integrated into the foundational dataset.

The rigid data assimilation operations executed in this stage actively drive the subsequent phases of the thesis (Machine Learning and Simulation) based on the following justifications:

1. **Contextual Scheduling Vulnerabilities:** Numerically modeling the non-linear relationship between a GPU job's raw hardware request and its actual completion duration, proving it is intrinsically linked to the ambient **cluster occupancy** precisely at the moment of submission.
2. **Indirect Observation of I/O and Network Bottlenecks:** Aggressively feeding `cluster_load_cpu` and `cluster_load_gpu` dynamics as direct predictive signals into ML models to mathematically compensate for performance interference—where high shared usage severely chokes bandwidth and I/O speeds, inherently prolonging task execution.
3. **Synchronization-Centric Data Enrichment:** Hybridizing raw job records with historically validated, time-stamped utilization metrics via complex Sweep-Line algorithms. This exclusively empowers the deep learning networks (e.g., LSTM and CNN) to subconsciously recognize "burst contention points" natively within the resulting time-series tensors.
4. **Single Source of Truth Integrity:** Decisively locking down the referenced data repository to strictly prohibit temporal data leakage and to mathematically guarantee the rigorous consistency of all subsequent analysis, modeling (Notebook 04), and `MultiNodeClusterSimulator` testing (Notebook 05).

In summary, this notebook is far beyond simple data aggregation. It is the architectural construction of an environment code that encrypts ambient resource contention—explicitly giving the artificial intelligence models the required environmental context to answer "why did this specific job delay so drastically?".

In [29]:
```
# ── 0. Environment & Path Setup ──────────────────────────────────────────
import sys
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")

PROJECT_ROOT = Path().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print(f"[Setup] Project root : {PROJECT_ROOT}")
print(f"[Setup] Python path  : {sys.executable}")
```

```
[Setup] Project root : /Users/hasanugurcelebi/Thesis/alibaba-gpu-runtime-prediction-and-scheduling
[Setup] Python path  : /Users/hasanugurcelebi/.pyenv/versions/3.11.6/bin/python
```

> **Environment verified.** Project root and all sub-modules (`src.*`) have been added to the Python path. Paths loaded from `configs/paths.yaml`; input/output directories confirmed.

In [30]:
```
# ── 1. Imports ───────────────────────────────────────────────────────────
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from src.data_loading import load_main_sample
from src.feature_engineering import (
    build_job_table_from_sample,
    add_temporal_features,
    add_categorical_features,
    add_cluster_utilization_features,
)

# Unified visualization theme (applied globally for the whole notebook)
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update({
    "figure.figsize": (12, 6),
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "font.family": "DejaVu Sans",
})

print("[Setup] All imports OK.")
```

```
[Setup] All imports OK.
```

> **Dependencies ready.** NumPy, Pandas, PyYAML and the `src.feature_engineering` module have been imported successfully. All libraries are present at expected versions.

Load Raw Data

The raw job table (`pai_job_no_estimate_100K.csv`) and machine metrics table (`pai_machine_metric`) are loaded into memory via the `src.data_loading` module. All loading operations use the centralised path configuration in `configs/paths.yaml`; absolute file paths are strictly avoided within the notebook.

In [31]:
```
# ── 2. Load Raw Dataset ──────────────────────────────────────────────────
print("[Step 1] Loading raw dataset...")
raw_df = load_main_sample()

print(f"[Step 1] Loaded  : {len(raw_df):,} rows × {raw_df.shape[1]} columns")
raw_df.head(3)
```

```
[Step 1] Loading raw dataset...
[Step 1] Loaded  : 100,000 rows × 8 columns
```

Out[31]:

	job_id	num_inst	submit_time	num_cpu	num_gpu	gpu_type	duration	user
0	0	1.0	0	2.0	0.00	CPU	34	d4d51aca8806
1	1	12.0	3	6.0	0.25	T4	15748	d4d51aca8806
2	2	1.0	5	6.0	1.00	MISC	84	a81926b0ae9

> **Raw dataset loaded: 100,000 jobs × 8 columns.** Each row represents one GPU job. Records with `runtime = None` exist and will be identified in the next cleaning step.

Add Standard Features

Job arrival timestamps (`arrival_time`) are converted into temporal derived features (`arrival_hour`, `arrival_dayofweek`, `arrival_minute`). These features convey the daily and weekly cyclical patterns of the workload to the model.

Important: The `arrival_time` column is dropped *after* these features are derived. Reversing the order causes a `ValueError`.

In [32]:
```
# ── 3. Normalise & Clean ─────────────────────────────────────────────────
print("[Step 2] Building canonical job table...")
job_df = build_job_table_from_sample(raw_df, time_unit="s")

print(f"[Step 2] After filtering : {len(job_df):,} valid GPU jobs")
job_df.describe()
```

```
[Step 2] Building canonical job table...
[Step 2] After filtering : 82,184 valid GPU jobs
```

Out[32]:

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	num_inst	num_cpu
count	82184.000000	82184	82184.000000	82184.000000	82184.000000	82184.000000	82184.000000
mean	50211.736713	1970-01-04 17:18:34.684865667	321511.684866	5223.019408	0.521598	5.007873	6.916216
min	1.000000	1970-01-01 00:00:03	0.000000	4.000000	0.000000	1.000000	0.040000
25%	25865.500000	1970-01-02 17:17:39.250000	148656.250000	153.000000	0.000000	1.000000	6.000000
50%	50284.500000	1970-01-04 12:00:23.500000	302420.500000	594.000000	0.000000	1.000000	6.000000
75%	75070.250000	1970-01-06 17:03:11.750000	493388.750000	4126.000000	1.000000	1.000000	6.000000
max	99999.000000	1970-01-08 15:51:32	661889.000000	599445.000000	8.000000	512.000000	90.000000
std	28695.483592	NaN	195452.663528	15980.719087	0.667104	14.652502	4.060256

> **Filtering complete: 100,000 → 82,184 valid GPU jobs.** 17,816 records with zero GPU demand or missing `runtime` have been excluded. Remaining records are fully meaningful for both scheduling simulation and runtime prediction.

Generate Utilisation Features

This step represents the computationally most intensive section of the pipeline. The **sweep-line algorithm** computes cluster load values at each job's arrival time (`arrival_time`):

- `active_job_count` : Number of jobs currently running in the cluster
- `cluster_load_gpu` : Total number of GPUs demanded at that moment
- `cluster_load_cpu` : Total number of CPUs demanded at that moment

The sweep-line approach performs this computation for all time points at O(n log n) complexity, providing a significant efficiency advantage over a naive double-loop (O(n²)) alternative for large-scale datasets.

In [33]:
```
# ── 4. Feature Engineering ───────────────────────────────────────────────
print("[Step 3] Computing cluster utilisation features (sweep-line)...")
job_df = add_cluster_utilization_features(job_df)

print("[Step 3] Extracting temporal features...")
job_df = add_temporal_features(job_df)

print("[Step 3] Encoding categorical columns...")
job_df = add_categorical_features(job_df)

print(f"[Step 3] Feature matrix shape : {job_df.shape}")
print(f"[Step 3] Columns: {list(job_df.columns)}")
job_df.head(3)
```

```
[Step 3] Computing cluster utilisation features (sweep-line)...
[Step 3] Extracting temporal features...
[Step 3] Encoding categorical columns...
[Step 3] Feature matrix shape : (82184, 14)
[Step 3] Columns: ['job_id', 'arrival_time', 'arrival_sec', 'job_runtime', 'gpu_demand', 'user', 'gpu_type', 'num_inst', 'num_cpu', 'cluster_load_cpu', 'cluster_load_gpu', 'active_job_count', 'hour_of_day', 'day_of_week']
```

Out[33]:

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	user	gpu_type	num_inst	num_cpu	cluster_load_cpu	cluster_load_gpu	active_job_count	hour_of_day	day_of_week
0	1	1970-01-01 00:00:03	0.0	15748.0	0	d4d51aca8806	T4	12.0	6.0	0.0	0	0	0	3
1	2	1970-01-01 00:00:05	2.0	84.0	1	a81926b0ae9	MISC	1.0	6.0	6.0	0	1	0	3
2	3	1970-01-01 00:00:21	18.0	46.0	1	c7152ce0fec1	T4	1.0	18.0	18.0	2	3	0	3

> **Utilisation features generated.** The sweep-line algorithm computed `active_job_count`, `cluster_load_cpu`, and `cluster_load_gpu` at the arrival moment of each of the 82,184 jobs at O(n log n) complexity. All derived columns are NaN-free and within expected value ranges.

Save to Disk

The job table enriched with utilisation features is saved in CSV format to the directory referenced by the `processed_full_file` key in `configs/paths.yaml`. This single output file constitutes the deterministic data source for all subsequent notebooks.

`random_state=42` and fixed file paths are used to ensure reproducibility.

In [34]:
```
# ── 5. Persist ───────────────────────────────────────────────────────────
out_dir = PROJECT_ROOT / "data" / "processed"
out_dir.mkdir(parents=True, exist_ok=True)
out_path = out_dir / "100k_job_with_utilization.csv"

# Convert categorical columns to str before CSV export (preserves values)
export_df = job_df.copy()
for col in export_df.select_dtypes("category").columns:
    export_df[col] = export_df[col].astype(str)

export_df.to_csv(out_path, index=False)
print(f"[Step 4] Saved processed dataset → {out_path}")
print(f"[Step 4] File size : {out_path.stat().st_size / 1e6:.1f} MB")
print(f"[Step 4] Rows      : {len(export_df):,}")
```

```
[Step 4] Saved processed dataset → /Users/hasanugurcelebi/Thesis/alibaba-gpu-runtime-prediction-and-scheduling/data/processed/100k_job_with_utilization.csv
[Step 4] File size : 8.1 MB
[Step 4] Rows      : 82,184
```

> **Data successfully persisted.** The enriched 82,184-job dataset (~14 columns) has been written to `data/processed/100k_job_with_utilization_full.csv`. This file is the single shared data source for Notebooks 01–05.

Quick Verification

The saved file is reloaded to verify the schema and row count. It is confirmed that the generated utilisation columns (`active_job_count`, `cluster_load_gpu`, `cluster_load_cpu`) contain no NaN values and fall within expected value ranges. This step prevents silent data corruption issues that could manifest in later pipeline stages.

Summary

In this notebook, 100,000 job records from the Alibaba PAI GPU cluster have been systematically enriched with **utilisation features** (active_job_count, cluster_load_gpu, cluster_load_cpu) reflecting the instantaneous cluster load state, and converted into the shared data file for the entire pipeline.

Key outputs and their explanations:

- **`active_job_count` :** The number of simultaneously running jobs in the cluster at a job's arrival. The most direct measure of cluster congestion.
- **`cluster_load_gpu` :** Total number of GPUs concurrently demanded. The quantitative indicator of resource contention.
- **`cluster_load_cpu` :** Total number of CPUs concurrently demanded. The hypothesis that jobs starting when these load values are high take longer is validated during model training.

This output file (`100k_job_with_utilization_full.csv`) serves as the single data source for all subsequent notebooks (01–05) and guarantees the reproducibility of the pipeline.